第 4-3 讲 • 从需求到设计（下）：写出来并验收

> **模块 4 • 学会改代码** ｜ 建议用时 **40 分钟**（阅读 12 + 动手 28）
> 前置要求：完成 4-1、4-2（你手上应该有一份改动清单）

本节目标

读完并做完这一讲，你能：

1. 照着设计清单**独立完成一个跨三层的真实功能**（契约 → 数据层 → 界面）。
2. 学会**"先让类型报错，再逐个填"**——把 TypeScript 当成一份自动待办清单用。
3. 理解 SQL 里**"让用户选排序"最安全的实现方式**（映射表，不是拼接）。
4. 用**验收清单**确认功能正确，而不是"看着差不多就行"。
5. 体验一次完整的**回归验证**（改完跑一整套检查）。

一、开工前的三件事

照着 4-1 讲的流程走，前面三步不能省。

1.1 ① 确认基线

```
cd "C:\Users\34123\Desktop\记账簿"
npm run typecheck 2>&1 | Select-String "error TS" | Measure-Object | Select-Object -Expand
```

预期输出 `0`。如果不是 0，**先停下来搞清楚为什么**——别在脏基线上动手。

1.2 ② 备份要改的文件

这次要改 3 个文件（4-2 讲列的清单）：

```
$files = @(
  'src\shared\types.ts',
  'src\main\db\repositories\records.repo.ts',
  'src\renderer\src\features\records\RecordsPage.tsx'
)
foreach ($f in $files) {
  $name = Split-Path $f -Leaf
  Copy-Item $f "$env:TEMP\backup-$name" -Force
  "已备份: $name"
}
```

为什么这三个：它们是这次要动的全部文件。**改坏了可以一键还原**：

```
# 还原（需要时再执行）
Copy-Item "$env:TEMP\backup-types.ts" src\shared\types.ts -Force
```

1.3 ③ 启动开发模式

```
npm run dev
```

保持它开着——改完就能立刻看到效果。

二、第一步：改契约（`types.ts`）

2.1 加一个受限的排序类型

打开 `src\shared\types.ts`，在 `RecordQuery` 定义**前面**加上：

```
/** 记录列表的排序方式：只允许这几种 */
export type RecordSort = 'date-desc' | 'amount-desc'
```

然后在 `RecordQuery` 里加一个字段（放在 `types` 附近比较合适）：

```
  /** 排序方式，默认按日期倒序 */
  sort?: RecordSort
```

2.2 为什么要用这种"联合类型"

这是本讲最重要的设计点。

对比两种写法：

```
// ❌ 危险写法
sort?: string
// 用户可以传 'amount-desc'，也可以传 'date; DROP TABLE records'

// ✅ 安全写法
sort?: RecordSort        // 只能是 'date-desc' 或 'amount-desc'
```

好在哪：

好处	说明
类型层面限死	传别的值，编辑器立刻标红，**根本进不了运行时**
能安全地"查表"	数据层可以用它去查一张映射表，**不用把任何用户输入拼进 SQL**
可扩展	以后加"按分类排序"，加一个枚举值 + 一行映射就行

2.3 存盘，看类型检查报什么

```
npm run typecheck
```

预期：**0 错误**（只加了一个类型和可选字段，不影响任何现有代码）。

这一步很重要：说明你的契约改动是**向后兼容**的——`sort` 是可选字段，所有现有代码不传它也能通过。

三、第二步：改数据层（`records.repo.ts`）

3.1 先找到要改的那一处

```
Select-String -Path src\main\db\repositories\records.repo.ts -Pattern "ORDER BY"
```

你要改的是列表查询那一处（真实代码）：

```
      ${where.sql}
      ORDER BY r.date DESC, r.created_at DESC
      LIMIT ? OFFSET ?`
```

问题：排序写死了。**目标**：根据 `query.sort` 决定。

3.2 加一张"排序映射表"

在文件里合适的位置（比如 `buildWhere` 函数附近）加上：

```
/**
 * 排序方式 → SQL 片段。
 *
 * 用"查表"而不是"把用户传的值拼进 SQL"：这是防注入的关键。
 * 用户能提供的只有一个枚举值，它在这里当 key 用；真正进 SQL 的永远是
 * 这张表里我们自己写好的固定字符串，任何用户输入都不会进入语句。
 */
const SORT_SQL: Record<RecordSort, string> = {
  'date-desc': 'r.date DESC, r.created_at DESC',
  'amount-desc': 'r.amount DESC, r.date DESC'
}
```

注意 `amount-desc` 的写法：`r.amount DESC, r.date DESC` —— **金额相同时按日期排。**

为什么：如果不加第二排序条件，**同金额的记录每次查询顺序可能不一样**（数据库不保证稳定顺序）。用户会看到"列表顺序莫名其妙变了"。**加上日期作为第二条件，顺序就稳定了。**

这是一个很容易忽略的细节，但真实用户会注意到。

3.3 改 SQL 那一行

把写死的 `ORDER BY` 改成从映射表取：

```
// 找到那行 ORDER BY，改成：
      ORDER BY ${SORT_SQL[query.sort ?? 'date-desc']}
```

注意 `?? 'date-desc'`：这就是"默认值"。

- 老代码不传 `sort` → `undefined` → `??` 取 `'date-desc'` → **行为和以前完全一样**
- 新代码传 `'amount-desc'` → 按金额排

`??` **是什么**：叫"空值合并运算符"，意思是"**如果左边是 null 或 undefined，就用右边的**"。

为什么不用 `||`：`||` 会把空字符串、0 也当成"空"。对字符串枚举来说，`??` 更精确。

3.4 检查类型（这里一定会报一个错，别慌）

```
npm run typecheck
```

预期报错：

```
src/main/db/repositories/records.repo.ts(82,24): error TS2304: Cannot find name 'RecordSor
```

这是正常的，因为你在 `repo.ts` 里用了 `RecordSort` 这个类型，但**这个文件还没有 import 它。**

我实测跑过这一步，确实报这个错。 处理方法：找到文件顶部的 import，把 `RecordSort` 加进**已有的那行**（别新加一行）：

```
// 改动前
import type {
  CalendarDay,
  LedgerRecord,
  RecordInput,
  RecordListItem,
  RecordListResult,
  RecordQuery,
  RecordType
} from '@shared/types'

// 改动后（只在末尾加一个）
import type {
  CalendarDay,
  LedgerRecord,
  RecordInput,
  RecordListItem,
  RecordListResult,
  RecordQuery,
  RecordType,
  RecordSort
} from '@shared/types'
```

再跑一次：

```
npm run typecheck
```

预期：0 错误。

> **这是一个很好的教学时刻**：TypeScript 报的错**直指问题**（文件名 + 行号 + 缺少什么）。
> **遇到报错不要慌，先读它说了什么**——下一讲（4-4）专门教这个。

3.5 现在功能已经能用了吗?

能用了，但还没有入口——因为界面还不知道有这个参数。

你可以先验证数据层是对的（这一点很专业）：

```
npm run selftest 2>&1 | Select-String "通过" | Select-Object -Last 1
```

预期：`88/88 通过`。

为什么这时候要跑：确认你改的 SQL 没把数据层逻辑弄坏。**如果这里就报错，说明改错了，别继续往下做界面。**

我实测验证过这个方案真的生效（在沙箱里插了三笔金额不同的账）：

检查	实测结果
按金额降序排列	✅ `999900 > 12345 > 500`
同金额顺序稳定（查两次一致）	✅ 都是 `2026-09-20, 2026-09-01`
同金额时按日期降序	✅ 日期大的在前
不传 `sort` 时退回按日期	✅ 行为与升级前一致

所以你照做，结果应该和这一样。

四、第三步：改界面（`RecordsPage.tsx`）

4.1 加一个状态

在组件里（其他 `useState` 附近）加：

```
const [sort, setSort] = useState<RecordSort>('date-desc')
```

逐部分解释：

部分	意思
`sort`	当前排序方式（2-4 讲的状态）
`setSort`	修改它的函数
`useState<RecordSort>('date-desc')`	初始值是按日期倒序 —— **和现在的行为一致**

4.2 把状态放进 query（关键一步）

找到那个 `useMemo`（4-2 讲带你看过）：

```
const query = useMemo(() => {
  const base = allTime ? {} : monthRange(month)
  return { ...base, keyword: keyword.trim() || undefined, limit: 500 }
}, [allTime, month, keyword])
```

改成：

```
const query = useMemo(() => {
  const base = allTime ? {} : monthRange(month)
  return { ...base, keyword: keyword.trim() || undefined, limit: 500, sort }
}, [allTime, month, keyword, sort])      // ★ 依赖数组加上 sort
```

★ 这一处最容易错：**依赖数组必须加上 `sort`**。

不加会怎样：用户点"按金额"，`sort` 状态变了、界面重绘了，**但 `query` 没有重新计算**（因为依赖数组没变），发出去的请求还是旧排序——**看起来"点了没反应"**。

4.3 加 UI 控件

在页面的操作区（搜索框附近）加一个排序选择。项目里有现成的 `Segmented` 组件（2-4 讲见过）：

```
<Segmented
  value={sort}
  onChange={(value) => setSort(value)}
  options={[
    { value: 'date-desc', label: '按日期' },
    { value: 'amount-desc', label: '按金额' }
  ]}
/>
```

先确认 `Segmented` 已经 import 了：

```
Select-String -Path src\renderer\src\features\records\RecordsPage.tsx -Pattern "import.*ui
```

如果没有，就加上：

```
import { Button, Card, Input, Modal, Segmented } from '../../components/ui'
```

另外别忘了 import 类型：

```
import type { Attachment, RecordListItem, RecordListResult, RecordSort } from '@shared/typ
```

4.4 存盘，看效果

切到记账簿窗口（dev 模式会自动刷新）。

预期看到：搜索框附近多了「按日期 / 按金额」的切换。

点「按金额」：列表应该重新排序，**金额最大的在最上面。**

五、验收（这一步决定你是不是专业）

"看着有效果"不等于"做对了"。 用下面这张清单逐项确认。

5.1 功能验收

#	检查项	怎么验	预期
1	默认是"按日期"	刷新页面（`Ctrl+R`）	选中「按日期」，列表顺序和以前一样
2	切到"按金额"	点「按金额」	金额大的排前面
3	**同金额顺序稳定**	反复切换"按日期/按金额"	同金额的记录顺序**不变**（这就是加了第二排序条件的作用）
4	转账也在列表里	观察一笔转账	**转账的金额也参与排序**（它也是记录，有 amount）
5	和搜索能一起用	输入关键词 + 切换排序	两个条件同时生效

5.2 回归验收

① 类型检查：

```
npm run typecheck
```

预期：0 错误。

② 数据层自检：

```
npm run selftest 2>&1 | Select-String "通过" | Select-Object -Last 1
```

预期：`88/88 通过`。

③ 页面冒烟（确认没把界面改崩）：

```
npm run smoke:pages 2>&1 | Select-String "页面冒烟"
```

预期：`18/18 干净且渲染出内容`。

④ 记录页确实渲染出来了（冒烟里已经包含，但可以单独看）：

```
npm run smoke:pages 2>&1 | Select-String "/records"
```

预期：看到 `CLEAN /records ...`。

5.3 验收清单的意义

你可能觉得"跑这么多太麻烦"。但请注意：

检查	耗时	它防的问题
typecheck	3 秒	字段拼错、类型传错
selftest	8 秒	**把数据层逻辑改坏了**
smoke:pages	60 秒	**把界面改崩了**（白屏）

总共 1 分多钟，覆盖三类最容易出的错。 而这些错如果漏到用户那里，代价是"用户以为软件坏了"。

这就是"专业"和"业余"的差别：不是写得快，而是**知道自己没写坏。**

六、如果你做错了（怎么办）

这是本讲很实用的一节——第一次做几乎一定会出点错。

6.1 常见错误与解法

症状	原因	怎么修
点了「按金额」没反应	`useMemo` **依赖数组没加** `sort`	补上 `sort`（4.2 节 ★ 那处）
顺序乱七八糟、每次刷新都不一样	映射表里少了第二排序条件	改成 `'r.amount DESC, r.date DESC'`
报错"找不到 Segmented"	忘了 import	从 `../../components/ui` 补 import
报错"找不到 RecordSort"	忘了 import 类型	从 `@shared/types` 补 import
列表变空	SQL 拼错了（比如 `${SORT_SQL[...]}` 写成了别的）	看终端报错，对照 3.3 节
默认排序变了	忘了写 `?? 'date-desc'`	补上默认值

6.2 改坏了想全部还原

```
Copy-Item "$env:TEMP\backup-types.ts" src\shared\types.ts -Force
Copy-Item "$env:TEMP\backup-records.repo.ts" src\main\db\repositories\records.repo.ts -For
Copy-Item "$env:TEMP\backup-RecordsPage.tsx" src\renderer\src\features\records\RecordsPage
npm run typecheck
```

这就是 1.2 节备份的用途。 备份花 5 秒，能让你**放心大胆地试。**

6.3 一个重要的心态

第一次改三层的功能，做错是正常的。 重点不是"一次做对"，而是：

1. **有备份**，所以敢试
2. **有验收清单**，所以知道对不对
3. **有报错可查**（下一讲 4-4 专门教读报错）

改代码的能力 = 敢改 + 能发现错 + 会修。不是"不出错"。

七、自测题

1. 为什么排序要用"映射表查表"，而不是把用户传的值拼进 SQL?

2. `?? 'date-desc'` **这行在做什么？为什么必须有它?**

3. `useMemo` **的依赖数组里不加** `sort` **会怎样?**

4. 为什么 `amount-desc` **的 SQL 要写成** `r.amount DESC, r.date DESC` **（两个条件）?**

5. 改完一个功能后，至少要跑哪三个检查？各自防什么?

答案与解析

1. 因为拼接用户输入会导致 **SQL 注入**（传 `date; DROP TABLE records` 就能删库）。 → 用映射表：用户只能提供**枚举值**（当 key 用），**真正进 SQL 的永远是我们自己写好的固定字符串。原则：让用户从允许的选项里挑，绝不拼接用户输入。**

2. 它给"没传排序参数"的情况提供**默认值**（按日期倒序）。 → **必须有它**，因为它保证**老行为不变**：现有代码都不传 `sort`，取默认值后行为和升级前完全一致（**向后兼容**）。

3. 用户切换排序时，`query` **不会重新计算**，发出去的请求还是旧排序 → **看起来"点了没反应"**。 → 原因：`useMemo` 只在依赖变化时重算。这是 React 常见的"闭包捕获旧值"问题。

4. 因为**同金额的记录，数据库不保证稳定顺序**——只按金额排，相同金额那几笔的顺序可能每次查询都不同，用户会看到"列表顺序莫名其妙变了"。 → 加上 `r.date DESC` 作为第二条件，顺序就稳定了。**这是很容易忽略但用户会注意到的细节。**

5. ① `typecheck`（3 秒）：防字段拼错、类型传错；② `selftest`（8 秒）：防**把数据层逻辑改坏**；③ `smoke:pages`（60 秒）：防**把界面改崩**（白屏）。 → 总共 1 分多钟，覆盖三类最常见的错误。**这就是"专业"和"业余"的差别：不是写得快，而是知道自己没写坏。**

八、术语表

术语	一句话解释	
联合类型	限定取值范围的类型，如 `'a' \	'b'`
映射表	用"查表"代替"拼接"，把枚举值换成固定 SQL	
`??`（空值合并）	左边是 null/undefined 时取右边的值	
向后兼容	新版本不改变旧行为	
依赖数组	`useMemo` / `useEffect` 的重算触发清单	
稳定排序	相同值的记录顺序每次都一致	
第二排序条件	主条件相同时用于定序的次条件	
验收清单	逐项确认功能做对了的检查表	
回归验证	确认"没破坏原有功能"的检查	

九、本讲与项目的对应

- **你改动的文件**：`shared\types.ts`、`db\repositories\records.repo.ts`、`features\records\RecordsPage.tsx`
- **本讲不影响数据库、不影响你的记账数据**（没有新字段、没有迁移）
- 想保留这个功能：什么都不用做 ｜ 想还原：用 6.2 节的命令
- 上一讲：4-2 从需求到设计（上） ｜ 下一讲：**4-4 出错怎么办：读懂报错**

讲义版本：v1 ｜ 对应项目版本：0.1.0